

**CODE
FIRST
GIRLS**



CFGDEGREE

FOUNDATION

**MODULE TWO: INTRODUCTION TO GIT &
GITHUB**



MODULE 2

GIT & GITHUB





MODULE 2 : GIT & GITHUB

SESSION OVERVIEW

- + Version Control
- + Git Repositories
- + Git Commands
- + Working with Branching
- + Pull Requests & Merge Conflicts

TOPIC 1: VERSION CONTROL





TOPIC OVERVIEW: **VERSION CONTROL**

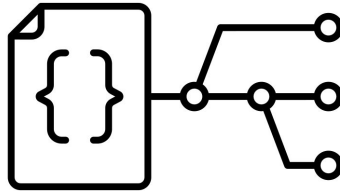
1. What is a version control system (VCS)?
2. What are the advantages of using a VCS?
3. How does version control work?
4. What is Git?
5. What is GitHub?



VERSION CONTROL SYSTEM (VCS)

LEARNING SESSION

A version control system is a system which allows us to track and manage changes to all types of files.



Version Control System
(VCS)



VERSION CONTROL SYSTEM (VCS)

LEARNING SESSION

Using a VCS has a number of advantages in software development:

- **Collaboration:** It allows team members to work together by managing changes to the code.
- **Version history:** It maintains a record of changes made over time.
- **Undo and rollback:** We can easily rollback/revert to a previous version of the code if a mistake or bug is detected.
- **Branching and merging:** This allows multiple people to implement changes on the same code, at the same time, without getting conflicts.



VERSION CONTROL SYSTEM (VCS)

LEARNING SESSION

- Imagine we are writing a document in Microsoft Word or Google Docs.
- We might save different versions like **Report_v1.docx**, **Report_Final.docx**, or even **Report_FINAL_FINAL.docx**.
- We might want to check what we wrote yesterday, compare two versions, or undo a mistake.
- Google Docs keeps a history of every change. We can see who changed what, when, and even go back to an old version if needed.
- This is exactly what a VCS does - but for code and other files, and in a much more powerful and organised way.



GIT

LEARNING SESSION

Git is a free, open source VCS that we can install locally on our personal computers.

- Git is the most popular and widely used version control system for developers.
- When we install Git, it provides a command-line interface for performing version control operations, such as creating repositories, committing changes, and managing branches.
- A command-line interface means we interact with Git by typing out commands, instead of a visual interface with buttons.



GITHUB

LEARNING SESSION

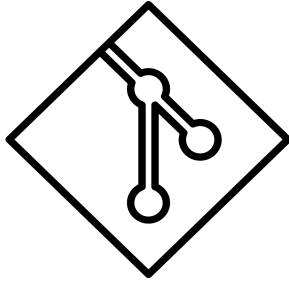
GitHub is a website where we can store and share Git projects.

- GitHub allows teams to share code, collaborate remotely, and manage projects using Git.
- There are plenty of other online services we can use to share Git projects, however GitHub is the one of the most popular and commonly used in industry.
- With GitHub, we are able to use the website, instead of doing everything by the command-line.



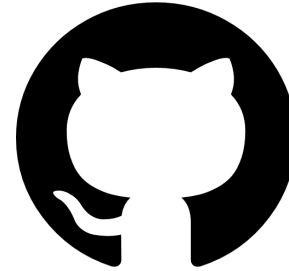
GIT & GITHUB SUMMARY

LEARNING SESSION



Git

- Version Control System
- Use on computers
- Tracks code



GitHub

- Website
- Share code and collaborate
- Stores online

TOPIC 2: GIT REPOSITORIES





TOPIC OVERVIEW: **GIT REPOSITORIES**

1. What is a repository?
2. What is a local repository?
3. What is a remote repository?



REPOSITORY

LEARNING SESSION

A repository stores the complete history of a project. It contains all the files, revision history, and other information about a project.

- We can think of a repository as a special folder where projects live, but it is much more powerful than just a folder.
- It tracks all the files in the project, as well as all the changes made to these files.
- Git has two types of repositories:
 - Local
 - Remote



GIT REPOSITORY

LEARNING SESSION

A local repository is a Git project stored on your own computer.

- A local repository is created when we initialise a folder as a Git repository.
- It tracks all our file changes and history locally.
- We don't need the internet to use it.
- Every developer working on a Git project will have their own local repository.



GIT REPOSITORY

LEARNING SESSION

The Remote Repository is a version of our project that is stored online, like on GitHub.

- It let's us back up work and collaborate with other developers.
- Each person working on a project can upload (push) their changes to the remote repository.
- And then other team members can download (pull) those changes to save them to their local repository.



README

LEARNING SESSION

A **README** is a text file that provides essential information about a project, including its purpose, setup instructions, and usage guidelines.

- It's usually the first file developers see when accessing a project.
- It includes what the project does and how to run it.
- Helps developers and users navigate and collaborate efficiently.
- We write README files in a syntax called [Markdown](#).



README

LEARNING SESSION

This is an example
of a README file

```
1  # CFG Week One
2  ## Intro to the world of the web and HTML
3
4  Welcome to Code First Girls!
5
6  To complete this exercise, click on the green `Clone or Download` button on the top right,
7  and select the `Download ZIP` option.
8
9  Unzip the folder and move it into your `coding_course` folder. Then, open the whole folder in Atom.
10
11 *Top tip! You can do this by dragging the folder onto the Atom icon on Macs*
12
13 ## Task one
14 Open the file `example.html` in Chrome and look around with the developer tools. What HTML tags can you see,
15 and what do you think they stand for?
16
17 ## Task two
18 Open the file `index.html` in your text editor and in Chrome, change it into valid HTML file so that it looks
19 like `solution.jpg`. Make sure to add the `doctype`, `html`, `head`, and `body` tags!
20
```



README

LEARNING SESSION

🔗 CFG Week One

Intro to the world of the web and HTML

Welcome to Code First Girls!

To complete this exercise, click on the green `Clone` or `Download` button on the top right, and select the `Download ZIP` option.

Unzip the folder and move it into your `coding_course` folder. Then, open the whole folder in Atom.

Top tip! You can do this by dragging the folder onto the Atom icon on Macs

And this is what it looks like when it's displayed in GitHub!

Task one

Open the file `example.html` in Chrome and look around with the developer tools. What HTML tags can you see, and what do you think they stand for?

Task two

Open the file `index.html` in your text editor and in Chrome, change it into valid HTML file so that it looks like `solution.jpg`. Make sure to add the `doctype`, `html`, `head`, and `body` tags!

TOPIC 3: GIT COMMANDS





TOPIC OVERVIEW: **GIT COMMANDS**

1. What are the main steps of the Git workflow?
2. What is **git init**?
3. What is **git add**?
4. What is **git commit**?
5. What is **git push**?
6. What is **git pull**?
7. What is **git clone**?



GIT WORKFLOW

LEARNING SESSION

The Git workflow involves three main steps:

1. Make code changes locally on your computer.
2. Use Git commands to track and save changes locally.
3. Push the changes to a remote repository, like GitHub, to share with others.

Let's explore these steps now, as well as the Git commands needed.



GIT COMMANDS: GIT INIT

LEARNING SESSION

git init will create a new, empty git repository in the current folder.

- We can turn any folder on our computer into a Git repository with this command

We start all of our Git
commands with **git**

→ **git** init



GIT COMMANDS: GIT ADD

LEARNING SESSION

git add tells Git which files we want to include in our next save.

- In order for Git to track files, we need to use the command: **git add**
- Once they are added, Git tracks them, and the changes made to them.

</> GIT COMMANDS: GIT ADD

LEARNING SESSION

`git add <filename>`

We can add specific files here

`git add .`

or use `.` to add all changed files



GIT COMMANDS: GIT COMMIT

LEARNING SESSION

git commit saves a snapshot of changes in the local repository.

- This command allows us to document changes, and add a message explaining what has been done.
- For example, if we have just built a new feature, our commit message might look like:

```
git commit -m "Add new feature"
```

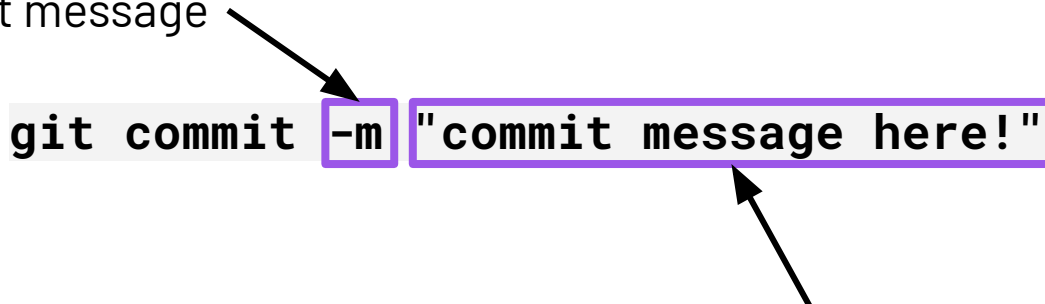
- When we commit, changes are saved locally, but not shared remotely.

</> GIT COMMANDS: GIT COMMIT

LEARNING SESSION

When we commit, we must add **-m** to add a commit message

```
git commit -m "commit message here!"
```



Add a commit message in double quotation marks



GIT COMMANDS: GIT PUSH

LEARNING SESSION

git push sends our changes from the local repository to the remote repository.

- We can make as many commits as we want locally, before deciding to push those changes.
- When we push, our changes go onto the remote repository, where changes are shared with the rest of the development team.
- The first time we push, we need to run the command **git push -u origin main**
- After the first push, we can run **git push**

</> GIT COMMANDS: GIT PUSH

LEARNING SESSION

We need to add this for the initial push

```
git push -u origin main
```

```
git push
```

Then, we just use **git push** after the initial push



GIT COMMANDS: GIT PULL

LEARNING SESSION

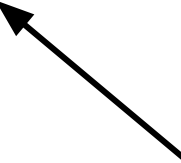
git pull brings down the latest changes from GitHub to our local repository.

- Whilst we are committing and pushing changes, another developer could be making their own changes too.
- We use **git pull** to get the most up-to-date version of the remote repository in our local repository.
- Always use **git pull** before starting a new piece of work, and before pushing our own changes.

</> GIT COMMANDS: GIT PULL

LEARNING SESSION

```
git pull
```



This will 'pull down' the latest changes, and copy them into the local repository



GIT COMMANDS: GIT CLONE

LEARNING SESSION

git clone is the process of making a copy of a remote repository onto a local machine.

- Sometimes, we want to work on a Git repository that has already been made and is hosted in GitHub, that we don't have a copy of locally.
- We can use git clone to get a copy of this repository.
- Find a repository to clone, e.g https://github.com/CFGer/demo_repo



GIT COMMANDS: GIT CLONE

LEARNING SESSION

The screenshot shows the GitHub interface for a repository named 'demo_repo' by user 'CFGer'. The repository is public and has 1 branch (main) and 0 tags. The 'main' branch is selected. The repository contains a single file, 'README.md', which was added by 'CFGer' in a commit '66e4adb' at 'now'. The 'README' file is open, showing the title 'README file' and the content 'This is a README for a demo CFG repository'. A purple callout bubble points to the repository name and says 'Open the repository in GitHub'.

CFGer / demo_repo

code Issues Pull requests Actions Projects Wiki Security Insights Settings

demo_repo Public

main 1 Branch 0 Tags

Go to file t Add file <> Code

CFGer Add README 66e4adb · now 1 Commit

README.md Add README now

README

README file

This is a README for a demo CFG repository

Open the repository in GitHub



GIT COMMANDS: GIT CLONE

LEARNING SESSION

CFGer / demo_repo

Type to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

demo_repo Public

Pin Unwatch 1

main 1 Branch 0 Tags

Go to file

Add file

<> Code

CFGer Add README 66e4adb · now 1 Commit

README.md Add README now

README

README file

This is a README for a demo CFG repository

Click on the green Code button



GIT COMMANDS: GIT CLONE

LEARNING SESSION

CFGer / demo_repo

Type / to search

code Issues Pull requests Actions Projects Wiki Security Insights Settings

demo_repo Public Pin Unwatch 1

main 1 Branch 0 Tags

Go to file t Add file <> Code

Local Codespaces

Clone ?

HTTPS SSH GitHub CLI

git@github.com:CFGer/demo_repo.git

Use a password-protected SSH key.

Open with GitHub Desktop

Download ZIP

Click copy

CFGer Add README

README.md Add README

README

README file

This is a README for a demo CFG repository



GIT COMMANDS: GIT CLONE

LEARNING SESSION

```
username  --zsh-- 61x11
Last login: Wed Apr  2 10:35:44 on ttys000
% git clone git@github.com:CFGer/demo_repo.git
```

Type git clone and
then paste the
repository in



GIT COMMANDS: GIT CLONE

LEARNING SESSION

```
username --zsh -- 61x11
Last login: Wed Apr  2 10:35:44 on ttys000
% git clone git@github.com:CFGer/demo_repo.git
Cloning into 'demo_repo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
(from 0)
Receiving objects: 100% (3/3), done.
% █
```

We now have a copy of the repository on our computer!

TOPIC 4:
WORKING WITH BRANCHING



TOPIC OVERVIEW: **WORKING WITH BRANCHING**

1. What is a git branch?
2. How can we view branches in GitHub?
3. What are some common Git commands in branches?



GIT BRANCHING

LEARNING SESSION

- The basic Git workflow we have learnt so far does work, but it has limitations when working on multiple features or collaborating in a team.
- What would happen if:
 - We're halfway through developing a new feature, but need to fix a critical bug?
 - Two developers accidentally overwrite each other's changes?
- Branching can help address these issues!



GIT BRANCHING

LEARNING SESSION

A branch is a separate line of development. It allows us to work on new features or bug fixes without affecting the main codebase.

- By default, everything we do in Git is on the **main** branch.
- We can 'branch off' to separate our work. This creates new branches.
- It is best practice to create a new branch each time we work on a new piece of work, issue/bug, or feature.
- After the work is complete on the branch, we need to merge it back into the **main** branch.



GIT BRANCHING

LEARNING SESSION



demo_repo

Public



feature-branch ▾



2 Branches



0 Tags

When we create a new branch, we can see it added here in GitHub

This branch is up to date with **main**.



GIT BRANCHING

LEARNING SESSION



demo_repo

Public



feature-branch ▾



2 Branches



0 Tags

This branch is up to date with **main**.

We can see it is up to date with the **main** branch



GIT BRANCHING

LEARNING SESSION

```
❯ README.md ×  
1  # README file  
2  This is a README for a demo CFG repository  
3  
4  # feature-branch update  
5  Adding some more info on my feature-branch  
6
```

Let's update our
README file



GIT BRANCHING

LEARNING SESSION



demo_repo

Public



Pin



Unwatch

1



feature-branch had recent pushes 3 seconds ago

Compare & pull request



feature-branch ▾



2 Branches



0 Tags

Go to file



Add file ▾

<> Code ▾

This branch is 1 commit ahead of main .



Contribute ▾

Now, we can see we
are 1 commit ahead
of the **main** branch



COMMON GIT COMMANDS ON BRANCHES

LEARNING SESSION

Create A New Branch

\$ git branch feature

Switch to an existing branch

\$ git switch feature

Create a new branch and immediately switch to it

\$ git checkout -b development

Delete a branch

\$ git branch -d feature

List all branches

\$ git branch

Create a branch with
git branch
<branch-name>



COMMON GIT COMMANDS ON BRANCHES

LEARNING SESSION

Create A New Branch

\$ git branch feature

Switch to an existing branch

\$ git switch feature

Move to that branch
with **git switch**
<branch-name>

Create a new branch and immediately switch to it

\$ git checkout -b development

Delete a branch

\$ git branch -d feature

List all branches

\$ git branch



COMMON GIT COMMANDS ON BRANCHES

LEARNING SESSION

Create A New Branch

```
$ git branch feature
```

Switch to an existing branch

```
$ git switch feature
```

Create a new branch and immediately switch to it

```
$ git checkout -b development
```

Delete a branch

```
$ git branch -d feature
```

List all branches

```
$ git branch
```

Create a new branch and move to it in the same command with **git checkout -b <branch-name>**



COMMON GIT COMMANDS ON BRANCHES

LEARNING SESSION

Create A New Branch

```
$ git branch feature
```

Switch to an existing branch

```
$ git switch feature
```

Create a new branch and immediately switch to it

```
$ git checkout -b development
```

Delete a branch

```
$ git branch -d feature
```

Delete a branch with

```
git branch -d  
<branch-name>
```

List all branches

```
$ git branch
```



COMMON GIT COMMANDS ON BRANCHES

LEARNING SESSION

Create A New Branch

\$ git branch feature

Switch to an existing branch

\$ git switch feature

Create a new branch and immediately switch to it

\$ git checkout -b development

Delete a branch

\$ git branch -d feature

List all branches

\$ git branch

View all branches
with **git branch**

TOPIC 5: PULL REQUESTS & MERGE CONFLICTS





TOPIC OVERVIEW: **PULL REQUESTS & MERGE CONFLICTS**

1. What is a pull request?
2. How do I create a pull request?
3. How do I merge pull requests?
4. What are 'fetching' and 'pulling'?



PULL REQUEST

LEARNING SESSION

- We have improved on our basic workflow by implementing branching, but how do we ensure code on a branch is ready to be merged into the **main** branch?
- We can introduce pull requests!



PULL REQUEST

LEARNING SESSION

A pull request is a request to review and merge your branch into another branch - usually the **main** branch.

- Think of a pull request like asking for feedback on your work, before adding it to the final version.
- Team members can review code, suggest changes, and approve it.
- Reviewing each other's code is a key part of the role of a software developer, improving code quality and reducing the number of issues and bugs.



PULL REQUEST

LEARNING SESSION

The workflow for using pull requests in Git are as follows:

1. Create a branch, make your changes, and commit.
2. Push your changes to the remote repository.
3. Open a Pull Request in GitHub to compare changes between your branch and the **main** branch.
4. Ask colleagues to review your code, and accept your changes.
5. Merge the Pull Request once it's been approved.



CREATING A PULL REQUEST

LEARNING SESSION

demo_repo Public Pin Unwatch 1

feature-branch had recent pushes 3 seconds ago **Compare & pull request**

feature-branch 2 Branches 0 Tags Add file <> Code

This branch is 1 commit ahead of main Contribute

Annotations:

- This is our branch from the previous topic (points to feature-branch)
- We can create a pull request here (points to Compare & pull request)



CREATING A PULL REQUEST

LEARNING SESSION

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main

compare: feature-branch

✓ Able to merge. These branches can be automatically merged.

Add a title

Update README

Add a description

WritePreview

Add your description here...

or click to add files

Create pull request

Reviewers

No reviews

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

When we open a pull request we can add a description

And request reviewers



CREATING A PULL REQUEST

LEARNING SESSION

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

🔗

base: main

⏪

compare: feature-branch

✓ Able to merge. These branches can be automatically merged.

Add a title

Update README

Add a description

Write

Preview

H B I ≡ <> 🔗 | ≡ ≡ ≡ | 📎 @ ↻ ↩

Add your description here...

📄 Markdown is supported

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

Click create

Create pull request



CREATING A PULL REQUEST

LEARNING SESSION

The screenshot displays a GitHub pull request interface. At the top, there are tabs for 'Conversation' (0), 'Commits' (1), 'Checks' (0), and 'Files changed' (1). A comment by 'CFGer' is visible, stating 'Updated the README file'. Below the comment, a commit titled 'Update README' is shown with a 'Verified' status and the commit hash 'a946687'. A green box indicates 'No conflicts with base branch' and 'Merging can be performed automatically'. A green button labeled 'Merge pull request' is highlighted with a purple border. A purple speech bubble points to this button, containing the text: 'When our changes have been approved by a reviewer, we can merge'.

Conversation 0 Commits 1 Checks 0 Files changed 1

CFGer commented now Owner ...

Updated the README file

Update README Verified a946687

No conflicts with base branch
Merging can be performed automatically

Merge pull request

You can also merge this with the command line. [view command line instructions.](#)

When our changes have been approved by a reviewer, we can merge



MERGING PULL REQUEST

LEARNING SESSION

Conversation 0

Commits 1

Checks 0

Files changed 1



CFGer commented 1 minute ago

Owner ...

Updated the README file



 Update README

Verified a946687



 CFGer merged commit **e19ad15** into `main` now



Pull request successfully merged and closed

You're all set — the `feature-branch` branch can be safely deleted.

Delete branch

Now we can see the pull request has been merged and closed.



MERGE CONFLICTS

LEARNING SESSION

A Git merge conflict is an error when Git cannot automatically merge changes.

- Merge conflicts are often encountered when working in a team, where multiple people have made changes to the same file, but on different branches.
- Merge conflicts require some human intervention to decide how best to combine or resolve the conflicts.
- Git puts all the changes together with different headers, so we can review what to keep.



MERGE CONFLICTS

LEARNING SESSION

We have to look through the file and decide which changes to keep or remove.

```
colors.txt x
src > colors.txt
1 red
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
2 <<<<<< HEAD (Current Change)
3 green
4 =====
5 white
6 >>>>>> his-branch (Incoming Change)
7 blue
```